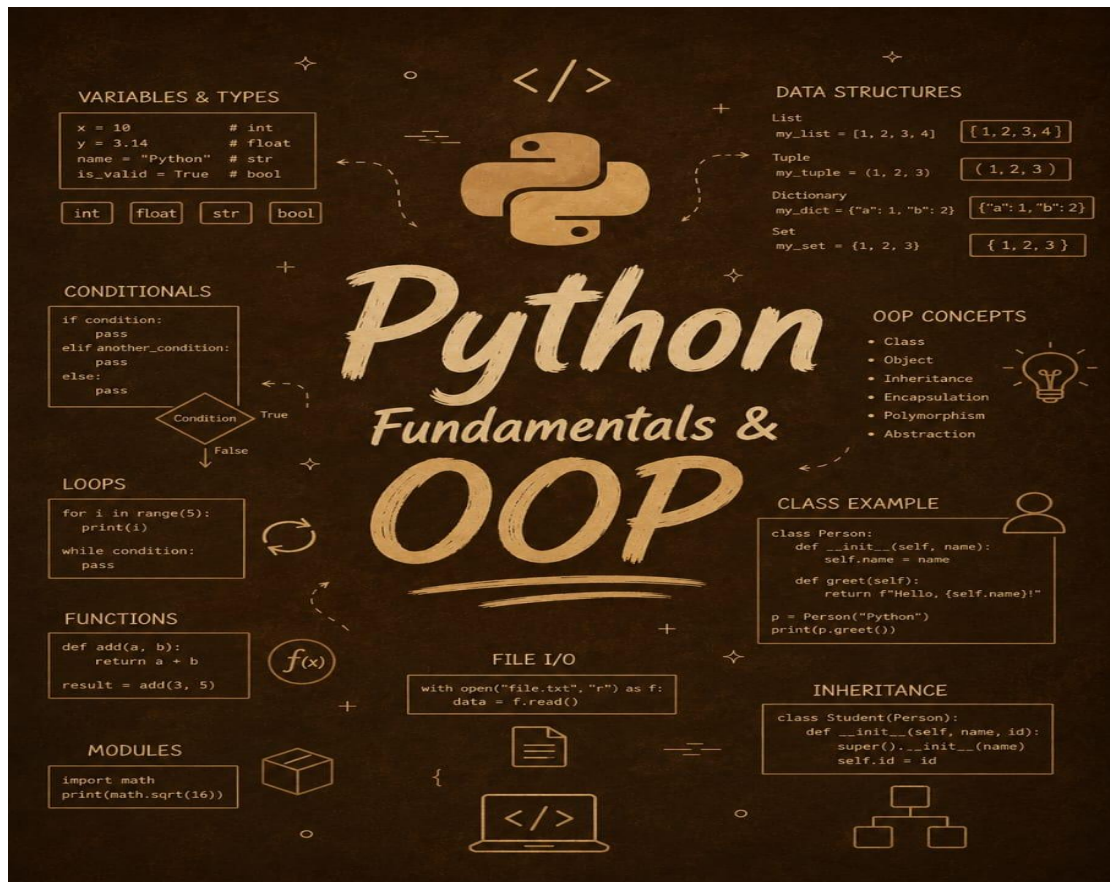




Python Fundamentals and Object Oriented Programming

By Jawad Jammoul

Content:

Chapter 01 : Syntax	Page
	1
Chapter 02 : Conditional Statements	20
Chapter 03 : Loops	24
Chapter 04 : Functions	31
Chapter 05 : Classes and Objects	35
Chapter 06 : Strings	41
Chapter 07 : Inheritance and Polymorphism	46
Chapter 08 : ArrayList	50
Chapter 09 : Exception Handling and Text I/O	53
Chapter 10 : Abstract Classes and Interfaces	57
Chapter 11 : Inner and Nested Classes in Python	61
Chapter 12 : Enum	64

Chapter 01 : Syntax

01*What is Python?

Python is a high-level , interpreted , and general-purpose programming language create by Guido van Rossum and first released in 1991

It is designed to be simple , readable , and powerful , making it one of the most popular programming languages in the world today

Easy to read and write

Open-source and free to use

Works on all major operating systems

Supports multiple paradigms(procedural , object-oriented, functional)

02*Why Using Python:

Python is widely used in different fields because of its simplicity and flexibility

Main uses of Python:

Web development -> (Django , Flask)

Data analysis -> (Pandas , NumPy)

Ai and Machine Learning -> (TensorFlow , PyTorch)

Automation and scripting

Game development

Cyber security and network programming

Advantages:

- 1.Readable syntax : looks almost like English
- 2.Large community : easy to find help and resources
- 3.Rich libraries : thousands of build-in and external modules

03*Print Stream:

.The print() function in Python is used to display output on the screen

Basic example:

```
print("Hello World")
```

Output:

```
Hello World!
```

You can also use escape sequences:

\n -> New Line: Used to move the text to a new line

Example:

```
print("Hello\nPython")
```

Output:

```
Hello
Python
```

\t -> Tab Space:

Used to add a horizontal tab space (like pressing the Tab key on the key keyword)

Example:

```
print("Name:\tJawad")
print("Age:\t21")
```

Output:

```
Name:  Jawad
Age:   21
```

\\" -> Double Quote:

Used to include double quotes inside a string that's already surrounded by double quotes

Example:

```
print("She said, \"Python is easy!\")
```

Output:

```
She said , "Python is easy!"
```

04*Data Types and Variables:

A variable is a name that stores a value in memory . In Python , you don't need to declare a variable type , it's automatically detected

Example:

```
x=10;
y=3.16;
name = "Jawad" #String
is-Active = True # Boolean
```

Common Data Types:

Type	Name	Example
Int	Integer	x=5
Float	Floating Point	y=3.17
Str	String	"Hello"
Bool	Boolean	True or False
List	Ordered , Changeable	[1,2,3]
Tuple	Ordered , Unchangeable	(1,2,3)
Set	UnOrdered , Unique element	{1,2,3}
Dict	Dictionaries(Key-value pairs)	{"name" : "Jawad" , "age" : 21}

Note :

You can check the type of any variable with print(type(x))

Like print(type(5)) -> <class 'int'>

05*Compiler/Interpreter:

Compiler	Interpreter
An interpreter takes a single line of code at a time	A compiler takes the entire program in one go
The compiler generates an intermediate machine code	The interpreter never produces any intermediate machine code
The compiler is used by programming languages such as C ,C++ , C# , Scale , Java ,etc.	An interpreter is used by programming languages such as Python , PHP , Perl , Ruby , etc.
The compiler is best suited for the production environment	An interpreter is best suited for a software development environment

06*Assigning Multiple Values:

You can assign multiple values to multiple variables in a single line , a process called unpacking

Example 1:

```
a, b=5,8  
print(a+b , a-b , a*b , a/b , a**b)
```

Output:

```
13 -3 40 0.625 390625
```

Example 2:

```
a , b = 10 , "Python"  
print(a)  
print(b)
```

Output:

```
10  
Python
```

07*Variable Operations:

You can perform various operations on variables depending on their types:

Example:

```
n , word = 4 , "Data"  
city = "Paris"  
print(n*word)  
print(word*2)  
print(word+city)  
print(word+ "" + city)  
print(city*3)  
# print(n + city) # Error: cannot add integer and string
```

Output:

```
DataDataDataData  
DataData  
DataParis  
Data Paris  
ParisParisParis
```

08*Comments in Python:

Comments are parts of the code that Python ignores when running the program. They're used to make the code easier to read and understand for humans.

Types of Comments:

1.Single-line comment - starts with #

```
radius = 5  
area = 3.14*radius**2 #This calculates the area of a circle
```

2.Multi-line comment - either multiple # lines or triple quotes """ or '''

```
#This section of code  
#print a welcome message  
#to the user  
print("Welcome")  
  
"""  
this is another way  
to write multiple-line comments  
using triple quotes  
"""  
  
print("Let's start coding!")
```

09*Variables operations:

Operation : Sum with Integer and String

In Python , you cannot directly add an integer to a string because they are different data types . You must first convert the integer to a string

Example:

```
a,b = 7 , "25"  
#print(a+b) # Error: cannot add int and str  
print(str(a) + b) # "725"  
print("a = " + str(a)) # "a=7"
```

This works because str() changes the integer into a string before concatenation

10*User Input:

Ask the user for input:

```
name=input("enter your name:")  
print(name)  
print(type(name))
```

Example 01:

```
x=input("Enter a number:")  
y=input("Enter a number: ")  
print(x*2)  
print(x+y)  
#print(x-y) error
```

Output:

```
Enter a number: 13  
Enter a number: 12  
1313  
1312
```

Example 02:

```
x=int(input("Enter a number: "))
y=int(input("Enter a number: "))
print(x*2)
print(x+y)
print(x-y)
```

Output:

```
Enter a number: 13
Enter a number: 12
26
25
1
```

11*Types of Errors in Python:**1.Syntax Errors:**

A syntax error happens when the code breaks Python's grammar rule. It means Python doesn't understand what you wrote

Example:

```
print("Hello")
```

Output:

```
Syntax Error : unexpected EOF while parsing
```

2.Runtime Errors:

A runtime error happens while the program is running. It means the syntax is correct, but something goes wrong during execution

Example:

```
x = 10/0
```

Output:

```
ZeroDivisionError: division by zero
```

3.Logical Errors:

A logical error happens when the program runs without crashing, but gives the wrong result because the logic is incorrect

Example:

```
x = 5
y = 10
print("Average:", x + y/2)
```

Output:

```
Average: 10.0
```

12*Strings:

String:

A string can contain letters , numbers , and symbols , it needs to be within quotes

Example:

name = "Machine Learning" or name = "Machine Learning"

Legal String:

A string may not span across multiple lines or contain a (") character

```
"This is not  
a legal String"
```

```
"This is not a "legal" String either"
```

A string can represent characters by preceding them with a backslash

(\t) : tab character

(\n) : new line character

(\") : quotation mark character

(\\): backslash character

String Quotation " ' ' ' :

-> There is no difference between using single quotes ' and double quotes "

-> We can also escaping characters using a slash \:

Example:

```
print("That's very clear")  
print('he said: "the weather is nice today".')  
print('I\'m a single string')  
print("He said:\\"Gd morning\\"!")
```

Output:

```
That's very clear  
He said "the weather is nice today".  
I'm a single string  
He said : "Gd morning"!
```

Concatenating String:

When you want to print a variable with a string , there is a better method than concatenating strings

Example:

```
x = "Hello World"  
print(x + ' Jawad')
```

Output:

```
Hello World Jawad
```


The % operator:

The % operator after a string is used to combine a string with variables

The % operator will replace a %s in the string with the string variable that comes after it

You need to use the same number of %s terms in a string as the number of variables in parentheses

Note: %s used for strings and %d for numbers

```
name = 'Rola'
age = 20
color = 'Blue'
print('Your name is %s , you are %d years old , your favorite color is %s' %(name,age,color))
```

Output :

```
Your name is Rola , you are 20 years old , your favorite color is Blue
```

```
name = "Jawad"
print(f "Hello , {name}!")
```

Same

```
name = "Jawad"
print("Hello %s"%name)
```

```
age = 22
print("I am %d years old" %age)
name = "Jawad"
age = 22
print("My name is %s and I am %d years old" %(name ,age))
```

Converting:

```
text = '456'
print(int(text))
print(int(text)+100)
```

Output:

```
456
556
```

```
num = 456
text = str(num)
print(type(text))
```

Output:

```
<class 'str'>
```

String & list Index:

We start counting index from 0

```
lst = ["milk", "chocolate", "Sugar"]
text = "Python"
print(text[3]) #h
print(lst[1]) # chocolat
print(lst[-1]) # Sugar
```

List & String operations:

```
lst1 = ["Apple", "chocolate", "Sugar"]
lst2 = ["Tomato", "Cucumber"]
lst1+=lst2
lst2*=2
print(lst1) # ["Apple", "Banana", "Orange", "Tomato", "Cucumber"]
print(lst2) # ["Tomato", "Cucumber", "Tomato", "Cucumber"]
```

Length:

```
lst1 = ["Apple", "Banana", "Orange"]
text = "\nI am a simple string"
print(len(text)) # 20
print(len(lst1)) # 3
```

Slicing:

Example 1:

```
text = "Python is great"
lang = ['Python', 'C++', 'CH', 'java', 'R']
first_six = text[0:6]
print(first_six)
starting_at_five = text[5:]
print(starting_at_five)
without_last_five = text[:-5]
print(without_last_five + '\n')
some_languages = lang[2:4]
print(some_languages)
without_last_one = lang[:-1]
print(without_last_one)
```

Output:

```
n is great
Python is
['CH', 'java']

['Python', 'C++', 'CH', 'java']
```

Example 2:

```

thanks = 'Thank You'
# to print the entire string
print(thanks)
# to print the primary character of the string
print(thanks[0])
# to print the 3rd to the 7th character of the string
print(thanks[2:7])
print(thanks[4:])
# to print the string twice
print(thanks * 2)
print(thanks + "\tAgain!")

```

Output:

```

Thank You
T
ank Y
k You
Thank YouThank You
Thank You  Again!

```

str[begin : end]

Slicing works with three arguments as well

str[begin:end:step]**Example:**

```

text = "This is an example about slicing"
print(text[:3]) # Tssn x p o in
lst = [0,1,2,3,4,5,6,7,8,9,10]
print(lst[:8:3]) # [0,3,6]

```

Index	Character
Index 0	T
Index 3	S
Index 6	S
Index 9	N
Index 12	X
Index 15	P
Index 18	
Index 21	O
Index 24	
Index 27	I
Index 30	N

String Methods:

`len()`

Returns the number of items in a string, list, etc

`len("hello") -> 5`

`list()`

Converts an iterable (like a string) into a list

`list("abc") -> ['a', 'b', 'c']`

`split()`

Splits a string into a list using a separator (default is space)

`"helloworld".split() -> ['hello', 'world']`

`index()`

Returns the position of the first occurrence of a value

`"apple".index('p') -> 1`

`startswith()`

Checks if a string starts with a specific value

`"hello".startswith("he") -> True`

`endwith()`

Checks if a string ends with a specific value

`"hello".endswith("lo") -> True`

`isLower()`

Checks if all letters in the string are lowercase

`"hello".islower() -> True`

`isUpper()`

Checks if all letters in the string are uppercase

`"HELLO".isupper() -> True`

`isdigit()`

Checks if the string contains only digits

`"12345".isdigit() -> True`

`lower()`

Converts all characters in the string to lowercase

`"HeLlo".lower() -> "hello"`

`upper()`

Converts all characters in the string to uppercase

`"hello".upper() -> "HELLO"`

`ord(text)`

Converts a string into a number

`ord("a") is 97`

`ord("b") is 98`

`chr(number)`

Converts a number into a string

```
a = "Hello World!"  
print(a.replace("H", "J")) # Jello World!
```

split()

Splits the string into substrings if it finds instances of the separator

```
a = "Hello,World!"  
print(a.split(",")) # ['Hello', 'World!']
```

Format:

As we learned , we cannot combine strings and numbers like this:

```
age = 36  
txt = "My name is John , I am " + age  
print(txt)
```

But we can combine strings and numbers by using the format() method !

Example 1:

```
age = 36  
txt = "My name is John , I am {}"  
print(txt.format(age)) #My name is John , and I am 36
```

Example 2:

```
price = 49  
txt = "The price is {} dollars"  
print(txt.format(price)) # The price is 49 dollars
```

Add more placeholders:

Example 1:

```
quantity = 3  
itemNo = 567  
price = 49  
myOrder = "I want {} pieces of item number {} for {:.2f} dollars."  
# or myOrder = "I want {0} pieces of item number {1} for {2:.2f} dollars."  
  
print(myOrder.format(quantity,itemNo, price)  
# I want 3 pieces of itemnumber 567 for 49.00 dollars
```

Example 2:

```
myOrder = "I have {carname} , it is a {model}"  
print(myOrder.format(carname= "Ford" , model = "Mustang"))  
#I have a Ford , it is a Mustang
```

13*List Methods:

append() : Adds an item to the end of the list

Example:

```
lst = [1,2]
lst.append(3) # [1,2,3]
```

clear() : Removes all items from the list

Example:

```
lst = [1,2,3]
lst.clear() # []
```

copy() : Returns a shallow copy of the list

Example:

```
lst = [1,2]
newlst = lst.copy() #[1,2]
```

count() : Returns the number of times a value appears

Example:

```
lst = [1,2,2,3]
lst.count(2) #2
```

extend() : Adds all items from another list to the end

Example:

```
lst = [1,2]
lst.extend([3,4]) # [1,2,3,4]
```

index() : Returns the first index of a value

Example:

```
lst = [1,2,3]
lst.index(2) #1
```

insert() : Inserts an itm at a specific position

Example:

```
lst = [1,3]
lst.insert(1,2) # [1,2,3]
```

pop() : Removes and returns the last item (or item at index)

Example:

```
lst = [1,2,3]
lst.pop() -> [1,2]
```

remove() : Removes the first occurrence of a value

Example:

```
lst = [1,2,2,3]
lst.remove(2) # [1,2,3]
```

reverse() : Reverses the list in place

Example:

```
lst = [1,2,3]
lst.reverse() # [3,2,1]
```

sort() : Sorts the list in ascending order (by default)

Example:

```
lst = [3,1,2]
lst.sort() # [1,2,3]
```

14*Sets Methods:

A set is another python data structure that is Unordered collection with no duplicated Set items are unchangeable, but you can remove add/or add items whenever you like

```
a = set('aioveoiaaeiou')
print(a) # a = {'a', 'i', 'e', 'u', 'o'}
b = set([1,1,1,1,1,2,2,2,2,2,3,3,3,3,3,3,3])
print(b) # b = {1,2,3}
c = set('bcokcbzo')
print(c) # c = {'z', 'c', 'b', 'k', 'o'}
print(a&c) # {'o'}
print(a | c) # {'z', 'u', 'k', 'a', 'e', 'b', 'c', 'i', 'o'}
print(a-c) # {'a', 'e', 'i', 'u'}
print(list(a)[2]) # 'a'
```

add(): Add an element to the set

Example:

```
a = {1,2}
a.add(3)
print(a) # {1,2,3}
```

copy() : Returns a shallow copy of the set

Example:

```
a = {1,2}
b = a.copy()
print(b) # {1,2}
```

difference() : Returns a set with elements in the first set but not in the second

Example:

```
a = {1,2,3}
b = {2,4}
print(a.difference(b)) # {1,3}
```

difference-update() : Remove elements in the second set from the first set

Example:

```
a = {1,2,3}
b = {2}
a.difference_update(b)
print(a) # {1,3}
```

discard() : Removes an element if it exists (no error if not)

Example:

```
a = {1,2}
a.discard(2)
print(a) # {1}
```

intersection() : Returns elements common to both sets

Example:

```
a = {1,2}
b = {2,3}
print(a.intersection(b)) # {2}
```

intersection_update() : Updates the set with the intersection

Example:

```
a = {1,2}
b = {2,3}
a.intersection_update(b)
print(a) # {2}
```

isdisjoint() : Returns True if sets have no elements in common

Example:

```
a = {1,2}
b = {2,3}
print(a.isdisjoint(b)) # True
```

issubset() :

Example:

```
a = {1,2}
b = {1,2,3}
print(a.issubset(b)) # True
```

issuperset():

Example:

```
a = {1,2,3}
b = {2}
print(a.issuperset(b)) # True
```

pop() : Removes and returns a random element

Example:

```
a = {1,2,3}
print(a.pop()) # (Random element)
```

remove() : Removes an element (throw error if not found)

Example:

```
a = {1,2}
a.remove(2)
print(a) # {1}
```

symmetric_difference() : Returns elements in either set , but not both

Example:

```
a = {1,2}
b = {2,3}
print(a.symmetric_difference(b)) # {1,3}
```

symmetric_difference_update() : Updates the set with symmetric difference

Example:

```
a = {1,2}
b = {2,3}
a.symmetric_difference_update(b)
print(a)
```


union : Returns all unique elements from both sets

Example:

```
a = {1,2}
b = {2,3}
print(a.union(b)) # {1,2,3}
```

update() : Updates the set with all elements from another set

Example:

```
a = {1,2}
b = {3}
a.update(b)
print(a) # {1,2,3}
```

15*Dictionary:

Items-Data Types:

Dictionary key must be of a type that is immutable

For example , you can use an integer , float , string , Boolean , tuple or dictionary key:

```
thisdt = {"brand" : "Ford" , "electric" : False , "year" : 1964 , "color" : ["red" , "white" , "blue"]} #Allowed
```

Duplicates Not Allowed:

Dictionaries cannot have two items with the same key:

Example:

_Duplicate values will overwrite existing values:

```
thisdt = {"brand" : "Ford" , "model" : "Mustang" , "year" : 1964 , "year" : 2020}
print(thisdt)
```

Output:

```
{'brand' : 'Ford' , 'model' : 'Mustang' , 'year' : 2020 }
```

Dictionaries-Add Items:

Dictionaries are mutable

Assigning to a key that does not exist adds an entry

```
Students = {}
Students[100] = 'Sami' students
[101] = 'Rima'
Students[102] = 'Rola'
Students thisdt = {"brand" : "Ford" , "model" : "Mustang" , "year" : 1964}
thisdt["color"] = "red"
print(thisdt) # {'brand' : 'Ford' , 'model' : 'Mustang' , 'year' : 1964 , 'color' : 'red' }
```

Dictionaries length:

To determine how many items a dictionary has , use the len() function

```
print(len(thisdt))
```

Update Dictionaries:

The update() method will update the dictionary with the items from the given argument

```
thisdt = {"brand" : "Ford" , "model" : "Mustang" , "year" : 1964}
thisdt.update({"year":2020})
print(thisdt)
dt = {"Owner" : "Rami" , "Branch" : "Beirut"}
thisdt.update(dt)
print(thisdt)
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2020}
{'brand': 'Ford', 'model': 'Mustang', 'Owner': 'Rami', 'Branch': 'Beirut' }
```

Del/clear for any iterable:

The del keyword can also delete the dictionary completely:

```
thisdt = { "brand": "Ford", "model": "Mustang", "Year": 1964 } # delete thisdt
print(thisdt)
# this will cause an error because "thisdt" no longer exists
```

The clear() method empties the dictionary:

```
thisdt = {"brand": "Ford", "model": "Mustang", "year": 1964}
thisdt.clear()
print(thisdt) # {}
```

Dictionaries Methods:

pop() : Removes the item with the specified key and returns its value

```
my_dict = {"a": 1, "b": 2}
value = my_dict.pop("a") # value = 1
```

update() : Updates the dictionaries with key-value pairs from another dictionary or iterable

```
my_dict = {"a": 1 }
my_dict.update( {"b": 2 } ) # {"a": 1, "b": 2}
clear() : Removes all items from the dictionary:
my_dict = {"a": 1, "b": 2}
my_dict.clear() # {}
```

keys() : Returns a view object containing all the keys in the dictionary

```
my_dict = {"a": 1, "b": 2}
print(my_dict.keys()) # dict_keys(['a', 'b'])
```

Example Get a list of the keys:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = car.keys()
print(x) # before the change
car["color"] = "white"
print(x) # after the change
```

values(): Returns a view object of all the values in the dictionary

```
my_dict = {"a": 1, "b": 2}
print(my_dict.values()) # dict_values([1, 2])
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = car.values()
print(x) # before the change
car["year"] = 2020
print(x) # after the change
```

get(): Returns the value of the specified key. Returns None if key is not found.

```
my_dict = {"a": 1}
print(my_dict.get("a")) # 1
print(my_dict.get("b")) # None
```

popitem(): Removes and returns the last inserted key-value pair as a tuple

```
my_dict = {"a": 1, "b": 2}
item = my_dict.popitem()
# item = ("b", 2)
```

copy(): Returns a shallow copy of the dictionary

```
original = {"a": 1}
new_dict = original.copy()
# {"a": 1}
```

items(): The items() method will return each item in a dictionary, as tuples in a list.

Get a list of the key:value pairs

```
car = {"brand": "Ford", "model": "Mustang", "year": 1964}
x = car.items()
dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
```

Get Items: Check if Key Exists

To determine if a specified key is present in a dictionary use the in keyword:

Check if "model" is present in the dictionary:

```
thisdt = {"brand": "Ford", "model": "Mustang", "year": 1964}
if "model" in thisdt:
    print(thisdt["model"])
    print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

Nested dictionary:

A dictionary can contain dictionaries, this is called nested dictionaries.

Create a dictionary that contain three dictionaries:

```
myfamily = {
  "child1": {"name": "Emil", "year": 2004},
  "child2": {"name": "Tobias", "year": 2007},
  "child3": {"name": "Linus", "year": 2011}
}
{'child1': {'name': 'Emil', 'year': 2004},
'child2': {'name': 'Tobias', 'year': 2007},
'child3': {'name': 'Linus', 'year': 2011}}
print(myfamily["child2"]["name"]) # Tobias
```

in and not in:

```
students = {100:'Sami',101:'Rima',102:'Rola'}
print(100 in students) # True
print(200 in students) # False
print(102 not in students) # False
```

type() Dictionary, Set...

From Python's perspective, Collections are defined as objects with the data type:

```
thisList = ["apple", "banana", "cherry"]
print(type(thisList)) # <class 'list'>
thisTuple = ("apple", "banana", "cherry")
print(type(thisTuple)) # <class 'tuple'>
thisSet = {"apple", "banana", "cherry"}
print(type(thisSet)) # <class 'set'>
students = {100:'Sami',101:'Rima',102:'Rola'}
print(type(students)) # <class 'dict'>
```

Iterator constructors:

Using the list() method to make a list:

```
thislist = list()
```

Using the tuple() method to make a tuple:

```
thistuple = tuple(("apple", "banana", "cherry"))
```

Using the set() method to make a set:

```
thisSet = set()
```

the dict() constructor to make a dictionary:

```
thisdict = dict(name="John", age=36, country="Norway")
```

16*Math Methods in Python:

In Python , math operations are done using the math module.
You must import it before using its functions

```
import math
```

After that , you can use any math function like this:
math.functionName(value)

1.math.sqrt(x) -> Square Root

Returns the square root of a number

```
import math  
print(math.sqrt(16))
```

Output:

```
4.0
```

2.math.pow(x,y) -> Power

Returns x raised to the power of y.

```
import math  
print(math.pow(2,3))
```

Output:

```
8.0
```

Note: You can also use the shortcut 2**3 in Python

3.math.floor(x) -> Floor

Rounds the number down to the nearest integer

```
import math  
print(math.floor(4.9))
```

Output:

```
4
```

4.math.ceil(x) -> Ceil

Rounds the number up to the nearest integer.

```
import math  
print(math.ceil(4.1))
```

Output:

```
5
```

5.math.pi -> Constant PI (3.141592)

Gives the value of PI

```
import math  
print(math.pi)
```

Output:

```
3.14159265358793
```

6.math.fabs(x) -> Absolute Value
Returns the absolute (positive) value

```
import math  
print(math.fabs(-5))
```

Output:

```
5.0
```

7.math.factoriel(x) -> Factoriel
Returns the factoriel of a number (x!)

```
import math  
print(math.factoriel(5))
```

Output:

```
120
```

8.math.log(x) -> Natural Logarithm (base e)

```
import math  
print(math.log(10))
```

Output:

```
2.302585092994046
```

9.math.degrees(x) & math.radians(x)
Convert between radians and degrees

```
import math  
print(math.degrees(math.pi)) # Radians -> Degrees  
print(math.radians(180)) # Degrees -> Radians
```

Output:

```
180.0  
3.141592653589793
```

Chapter 02 : Conditional Statements

01*Blocks and indentation:

.Before we talk about conditions , we have to introduce blocks and indentation

.A block in programming consists of one or more statements grouped together to do a sequential actions

.A program can be seen as a block , which consists of statements and other nested blocks

.In other languages we used "begin" and "end" or {} to separate the blocks , in python , we use indentation (a tab or four whitespaces)

02*if Statements:

Use if to execute a block of code only if a condition is True

```
if condition:  
    #code block
```

- .The condition must return True or False
- .If it's True , the indented block will execute
- .If it's False , nothing happens

Example:

```
x = 10  
if x > 5:  
    print("x is greater than 5")  
b = True  
if b :  
    print("True")
```

03*if-else Statement:

Use if-else when you want to run one block if the condition is True , and another block if it is False

```
if condition:  
    # block if condition is True  
else:  
    # block
```

04*Nested if Statements:

You can place an if statement inside another if or else to check multiple levels of conditions

```
if condition1:  
    if condition2:  
        #block if both are True
```

The second if only runs if the first condition is True

05*Chained Conditions:

Use elif (short for "else if") when you need to check multiple different conditions one after the other

```
if condition1 :  
    #block  
elif condition2 :  
    #another block  
elif condition3 :  
    #another block  
else:  
    #final block if none are True
```

06*Single Statement Conditions:

You can write if , or even if-else , in a single line for short and simple conditions

If condition : action

Examples:

```
x = 7  
if x%2==1 : print("Odd")  
x = 10  
print("Even") if x%2==0 else print("Odd")
```

07*Boolean DataType:

.The bool() function allows you to evaluate any value , and give you True or False in return

.Evaluate a string and a number:

Example:

```
x = "Hello"  
y = 15  
print(bool(x))  
print(bool(y))
```

Output:

```
True  
True
```

08*Evaluate Values and Variables:

In python , there are many values that are evaluated as False:

x=0 , y=None , b=False , dt={} , lst=[] , t=()

09*Indentation:

What is the output for each code?

```
x = 30  
y = 34  
if(x < 20):  
    print('hello')  
    print(y)
```

Output:

```
x = 30  
y = 34  
if(x < 20):  
    print('hello')  
    print(y)
```

Output:

```
34
```

10*The not keyword:

The not keyword is a logical operator , and is used to reverse the result of the conditional statement:

Test if a is NOT greater than b:

```
a = 33  
b = 200  
if not a > b:  
    print("a is NOT greater than b")
```

Output:

```
a is NOT greater than b
```


11*Pass statement:

If statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error

```
a = 33
b = 200
if b > a:
    pass
```

12*Short Hand If:

If you have only one statement to execute, you can put it on the same line as the if statement

```
if a>b:print("a is greater than b")
```

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line

```
a = 2
b = 330
print("A") if a>b else print("B")
```

You can also have multiple else statements on the same line:

One line if else statement, with 3 conditions:

```
a = 330
b = 330
print("A") if a>b else print("=") if a==b else print("B")
```

13*Comparison Operators and Logical Operators :

Comparison Operators:

Comparison operators are used to compare two values
The result is always True or False.

==: equal to Example : 5 == 5 --> True	<: less than Example : 2 < 7--> True
!= : not equal to Example: 5 != 3 --> True	>=: greater than or equal Example : 4 >= 4 --> True
> : Greater than Example: 10 > 3 --> True	<=: less than or equal Example : 8 <= 5 --> False

Example:

```
x = 10
y = 5
print(x == y) # False
print(x>y) #True
print(x<=10) #True
```

Output:

```
False
True
True
```

Logical Operators:

Logical operators are used to combine multiple conditions

And -> True if both conditions are True

Example: (x>5 and y<10) -> True

Or -> True if at least one condition is True

Example: (x>5 or y>10) -> True

Not -> Reverses the result

Example: not(x>y) -> False

14*match-case in Python:

The match-case statement was added in Python 3.10

It works like a switch statement in other languages

It checks the value of a variable and runs the matching case

Example:

```
day = 4
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case _:
        print("Invalid day")
```

Chapter 03 : Loops

01*Why do we use Loops?

Loops are used in programming to repeat a block of code multiple times without writing it over and over again

They help us save time , reduce code size , and make programs more flexible

Example:

Let's say we want to print numbers from 1 to 5

Without a loop , we would have to write:

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

But with a loop we can make Python do it automatically

```
i = 1
while i <= 5:
    print(i)
    i+=1
```

Output:

```
1
2
3
4
5
```

i = 1 -> Starting point

while i <= 5: -> repeat the block as long as the condition is true

print(i) -> prints the current number

i+=1 -> Increases i by 1 each time

02*while Loops:

A while loop will run the block of statements written after it as long as the condition is evaluated as True:

```
while(<condition>):  
    <Block of statements>
```

Example:

```
i=0  
while(i<3):  
    print("hello")  
    i+=1
```

Output:

```
hello  
hello  
hello
```

03*for loops:

The for loops is an iterator that allows a code block to be repeated a certain number of times

```
for <variable> in <sequence>:  
    <statements>
```

Example:

```
for i in range(0,3):  
    print("hello")
```

Output:

```
hello  
hello  
hello
```

04* Iterate Strings & Collections:

```
name = "Rima"  
for letter in name:  
    print(letter)  
fruits = ["apple", "banana", "cherry"]  
for item in fruits:  
    print(item)  
thisTuple = ("apple", "banana", "cherry")  
for i in thisTuple:  
    print(i)  
thisSet = {"apple", "banana", "cherry"}  
for fruits in thisSet:  
    print(fruits)  
students = {100:'Sami',101:'Rima',102:'Rola'}  
for key,value in students.items():  
    print(key,value)
```

Output:

```
Plain text
R
i
m
a
apple
banana
cherry
apple
banana
cherry
100 Sami
101 Rima
102 Rola
```

Examples:

```
lang = ['Python','C++','C#','java','R']
l = "Python"
if l in lang:
    print("Found")
for i in lang:
    if(l == i):
        print("Found")
```

Output:

```
Found
Found
```

05* Range() function:

- The range function is used to iterate over a sequence of numbers
- range(begin, end) it iterates from the number starting with begin and ending with one less than the number end. Default the beginning is 0 if it's omitted.
- range(begin, end, step) the same concept but with an increment value step; we could use a negative step to go backward.

Example:

```
for i in range(10):
    print(i)
#multiplication by 2
for i in range(0,10):
    print(i*2)
#2 power index
for i in range(0,5):
    print(2**i)
#index power 2
for i in range(0,5):
    print(i**2)
print("even")
for i in range(0,15,2):
    print(i)
print("odd")
for i in range(1,15,2):
    print(i)
print('backward')
for i in range(10,0,-1):
```

```
print(i)
#end
for i in range(1,10,2):
    print(i,end=';')
for i in range(5,0,-1):
    print(i,end=';')
```

Output:

```
0
1
2
3
4
5
6
7
8
9
0
2
4
6
8
0
2
4
6
8
1
4
9
16
even
0
2
4
6
8
10
12
odd
1
3
5
7
9
11
13
backward
10
9
8
7
6
5
4
3
2
1
1,3,5,7,9,5,4,3,2,1,
```

06*Python statements for loops:

break : Jumps out of the closet enclosing loop

continue : Jumps to the top of the closest enclosing loop

pass : Does nothing , empty statement placeholder

```
for i in range(10):  
    if(i==5):  
        break  
    print(i)
```

Output:

```
1  
2  
3  
4
```

```
for i in range(10):  
    if(i%2==0):  
        continue  
    print(i)
```

Output:

```
1  
3  
5  
7  
9
```

The pass Statement:

```
for x in [0,1,2]:  
    pass
```

Output:

```
#having an empty for loop like this , would raise an error without the pass statement
```

07*else:

The else statement with a loop is a special statement for python

It's executed only if a break is not executed

```
i = 0  
while(i<3):  
    x =int(input("Guess the number:"))  
    if(x==9):  
        print("You win!")  
        break  
    i+=1  
else : print("Try again!")
```

Output 1:

```
Guess the number : 7  
Guess the number : 9  
You win!
```

Output 2:

```
Guess the number : 3
Guess the number : 7
Guess the number : 0
Try again !
```

08*Nested Loops:

A nested loop is a loop inside another loop

They are useful when you need to repeat something multiple times for each iteration of another loop

```
i=1
while l<= 3:  #outer loop
    j=1
    while j<=2: #inner loop
        print(f"l={l} , j={j}")
        j+=1
    i+=1
```

Output:

```
i=1 , j=1
i=1 , j=2
i=2 , j=1
i=2 , j=2
i=3 , j=1
i=3 , j=2
```

The outer loop (l) runs 3 times

For each iteration of i , the inner loop(j) runs 2 times

The inner loop completes all its iterations before the outer loop moves to the next i

Nested for Loop:

```
for i in range(1,4) : #outer loop
    for j in range(1,3) : #inner loop
        print(f"i={i} , j={j}")
```

Output:

```
i=1 , j=1
i=1 , j=2
i=2 , j=1
i=2 , j=2
i=3 , j=1
i=3 , j=2
```

Range(1,4) -> numbers 1 ,2 ,3

Range(1,3) -> numbers 1, 2

The inner loop runs fully for each value of the outer loop , same behavior as the while example

09*String & Collection Comprehension:

Comprehension lets you create new lists , sets , or other collections from existing iterables in a single , readable line

The general syntax for list comprehension is:

```
newlist = [ expression for item in iterable if condition]
```

Expression: The value or transformation to include in the new collection

Condition: An optional filter - only items where the condition is True are included

Original data remains unchanged

Examples with Lists:

```
#filter : keep all fruits except "apple"
fruits = [ "apple" , "banana" , "cherry" , "kiwi" , "mango"]
newlist = [fruit for fruit in fruits if fruit != "apple"]
# ["banana" , "cherry" , "kiwi" , "mango"]
# Numbers from 0 to 9 , but keep only those <5
nums = [n for n in range(10) if n<5]
#[0,1,2,3,4]
#Transform : convert all fruit names to uppercase
upper_fruits = [fruit.upper() for fruit in fruits]
# ['APPLE' , 'BANANA' , 'CHERRY' , 'KIWI' , 'MANGO']
#Conditional transformation
replace_banana = [fruit if fruit != "banana" else "orange" for fruit in fruits]
# ['apple' , 'orange' , 'cherry' , 'kiwi' , 'mango']
#label even/odd
labels = ["Even" if l%2 ==0 else "Odd" for l in range(8)]
#['Even' , 'Odd' , 'Even' , 'Odd' , 'Even' , 'Odd' , 'Even' , 'Odd']
#Squares of numbers
squares = [n*n for n in range(10)]
# [0,1,4,9,16,25,36,49,64,81]
#replace negative prices with 0
original_prices = [1,25 , -9.45 , 10.22 , 3.78 , -5.92 , 1.16]
prices = [p if p>0 else 0 for p in original_prices]
# [1.25 , 0 , 10.22 , 3.78 , 0 , 1.16]
```

Using Set Comprehension:

```
quote = "life , uh , finds a way"
unique_vowels = {ch for ch in quote if ch in 'aeiou'}
# {'a' , 'e' , 'u' , 'i' }
```

Set comprehensions work like list comprehensions but return unique unordered values

10*Dictionary Comprehensions:

```
squares = {i : i*i for i in range(8) }
# ==> {0: 0 , 1:1 , 2:4 , 3:9 , 4:16 , 5:25 , 6:36 , 7:49}
```

The expression is the current item in the iteration , but it is also the outcome , which you can manipulate before it ends up like a list item in the new list:

Set the values in the new list to upper case:

```
fruits = ["apple" , "banana" , "cherry" , "kiwi" , "mango"]
newlist = [x.upper() for x in fruits]
# ['APPLE' , 'BANANA' , 'CHERRY' , 'KIWI' , 'MANGO']
```


You can set the outcome to whatever you like:

Set all values in the new list to 'hello' :

```
fruits = "apple", "banana", "cherry", "kiwi", "mango"]
newlist = ['hello' for x in fruits]
print(newlist)
newlist = [x if x!= "banana" else "orange" for x in fruits]
print(newlist)
```

```
['hello', 'hello', 'hello', 'hello', 'hello']
['apple', 'orange', 'cherry', 'kiwi', 'mango']
```

Chapter 04 : Functions

01*Functions Introduction:

What is a Function?

A function is a block of code that performs a specific task

Instead of writing the same code many times , we can define a function once and reuse it whenever we need

Why Use Function?

- 1.To make the code organized and readable
- 2.To reuse code instead of repeating it
- 3.To divide a big program into smaller , easier parts
- 4.To make debugging and testing easier

```
def function_name (list of parameters) :
    statements
```

02*Parameter & Arguments:

```
def power(a,b) :
    print(b**a)
power(3,4)
```

In the first block of code , we wrote the script of a function named power , then we called this function by given her name with the appropriate parameters;

The output for this script will be : 3 power 4

The function could takes no parameters to one or many parameters . These parameters is called arguments when we call(invok) the function

03*Return multiple value:

In the second example above , the sum function returned the sum of two parameters a and b
Further , we can return more than one value; the output is a tuple of the values returned:

```
def Sum_Dif(a,b):
    s= a+b
    d = a-b
    return s ,d
print(Sum_Dif(12,24))
```

Output:

```
(36 , -12)
```

04*Default and keyword parameters:

Functions can have default or optional parameters , in a way that we can call them without given any arguments , if the argument is not given , the default value is used

```
def hello(name , h= "Good Morning");  
    print("%s%s!"%(h,name))  
hello("Rabih")  
hello("Rabih" , "Bonjour")
```

Output:

```
Good Morning Rabih!  
Bonjour Rabih!
```

First we called "hello" with one argument ; the value of the second argument was the default value
In the second call we overwrite the default value with "Bonjour" .

05*Keyword parameters:

Keyword parameters , same concept but when we call the function we write the argument as it was named plus the value

In this case , if we have more than one keyword we can call them in any order:

```
def hello(name , h= "Good Morning" , g= "Have a nice day" ):  
    print("%s%s, %s!" %(h , name , g))  
hello("Rabih")  
hello("Rabih" , g ="Happy Birthday" , h="Bonjour")
```

Output:

```
Good Morning Rabih , Have a nice day!  
Bonjour Rabih, happy Birthday!
```

06*Arbitrary number of Arguments * args:

We could have a situation that we can't know the exact number of arguments a function will have:

In this case , we put an arbitrary parameter(treated as a list) , then we call the function with whatever number of arguments:

```
def Sums(*i):  
    s = 0  
    for i in I :  
        s = s+ i  
    return s  
print(Sums(12,45,100))  
#lst = [23 , 56 , 1000]  
#print(Sums(lst)) (This statement will give an error)
```

07*Arbitrary keyword Arguments **Kwargs :

If you do not know how many keyword arguments that will be passed into your function.

Add two asterisk ** before the parameter name in the function definition

```
def employee(**kwargs):  
    for key , value in kwargs.items():  
        print("%s is %s" %(key,value))  
employee(First_name = "Mary" , Last_name = "Smith" , address = "London")  
print('Another employee')  
mydictionary = {"First Name" : "Dani" , "Last Name" : "Harb"}  
employee(**mydictionary)
```

Output:

```
First_name is Mary
Last_name is Smith
Address is London
First Name is Dani
Last Name is Harb
```

08*Global and Local:

The scope of a variable declared within the body of a function is the function itself;
It can't be used outside the function unless we declare it as a global variable

```
def test() :
    s = 8
    print(s)
    d = s+8
def test():
    s = 8
    print(s)
test()
d = s+ 8
print(d)
```

Output:

```
8
16
```

09*Lambda Function:

A lambda is a small anonymous function (function without a name)
It can take any number of arguments , but it can only contain one expression

Examples:

```
#Add two numbers
f = lambda x , y: x+y
print(f(2,3)) #5
#Lambda stored inside a list
lst = ['one', lambda x:x*x , 3]
print(lst) #16
#Square and Cube using lambda
square = lambda x : x*x
cube = lambda val : val**3
print("Square of 2:", square(2)) #4
print("Cube of 4:", cube(square(2))) # 64
```

10*Common Built-in Functions in Python:

1.print() -> Displays text or values

```
print("Hello , World!") # Output : Hello , World!
```

2.int() -> Converts a value into an integer

```
num = int("10")
print(num+5) #15
```

3.str() -> Converts a value into a String:

```
age = 20
print("Age: "+str(age)) # Age : 20
```

4.range() -> Generates a sequence of numbers

```
for i in range(3)
    print(i) # 0, 1, 2
```

5.type() -> Returns the data type of a variable

```
x = 3.14
print(type(x)) # <class 'float'>
```

6.help() -> Shows documentation for objects/functions

```
help(str) # Shows info about string methods
```

7.list() -> Creates a list from an iterable

```
chars = list("abc")
print(chars) # ['a', 'b', 'c']
```

8.dict() -> Creates a dictionary

```
d = dict(name = "John", age = 25)
print(d) # {'name': 'John', 'age': 25}
```

9.set() -> Creates a set (unique unordered elements)

```
s = set([1,2,2,3])
print(s) # {1,2,3}
```

10.bool() -> Converts a value into True or False

```
print(bool(0)) #False
print(bool(5)) #True
```

11.bin() -> Converts an integer into a binary string

```
print(bin(8)) #0b1000
```

12.complex() -> Creates a complex number

```
z = complex(2,3)
print(x) # (2+3j)
```

11*Map Function:

Map() function returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable (list, tuple, etc.)

Syntax: map(function, iter)

```
str_nums = ["4", "8", "6", "5", "3", "2", "8", "9", "2", "5"]
int_nums = map(int, str_nums)
list(int_nums)
```

Output:

```
[4,8,6,5,3,2,8,9,2,5]
```

```
first_it = [1,2,3]
second_it = [4,5,6,7]
list(map(pow, first_it, second_it))
```

Output:

```
[1,32,729]
```

```
String_it = ["processing", "strings", "with", "map"]  
list(map(str.capitalize, string_it))  
list(map(str.upper, string_it))
```

Output:

```
['PROCESSING', 'STRINGS', 'WITH', 'MAP']
```

Chapter 05 : Classes and Objects

01*Definition of a Class, Object, Constructor:

Class : A blueprint for creating objects

Object : A Specific instance of a class

Constructor : A special method to initialize an object ; in Python it's called `__init__()`

Notes:

The constructor is optional , if not defined Python creates default constructor

Attributes are variables that belong to the object and are defined inside the constructor or later

Methods are functions inside the class that can perform actions using the object's attributes

02*Creating an Object:

Use the class name followed by parentheses to create an object

If the class has constructor , pass required arguments

```
objectName = className()
```

03*Accessing an Object's Data and Methods:

Use the dot. Operator to access an object's attributes or methods

Notes:

Object.attribute -> access or modify attribute

Object.method() -> call a method

Methods can use self to refer to the current object

04*Object Creating and Attribute Assignment:

Attributes can be assigned after creating an object

Notes:

Python allows dynamic assignment of new attributes anytime

Attributes can also start as None (null) and be updated later

05*Reference Data Fields and the Null Value:

An attribute can reference another object or start as None

Multiple objects can reference the same data value if assigned

None is Python's equivalent of a null value

Example : Circle Class

```
class Circle :
def __init__(self , radius=None): # constructor
    self.radius = radius #Attribute
def area(self):    #Method
    if self.radius is not None:
        return 3.14 * self.radius**2
    else:
        return None
#Creating objects
circle1 = Circle()    #radius is None
circle2 = circle(5)   # radius = 5
#Assigning attribute later
circle1.radius = 7
#Accessing data and methods
print(circle1.radius) #7
print(circle2.radius) #5
print(circle1.area()) # 153.86
print(circle2.area()) # 78.5
#Reference data fields
circle3 = Circle()
circle3.radius = circle1.radius
print(circle3.radius) #7
```

Output:

```
7
5
153.86
78.5
7
```

06*Differences Between Primitive Types and Reference Types:

Primitive types: stores the actual value(int , float , bool , str)

Reference types: stores a reference (memory address) to the object (list , class objects , dict , etc)

Example:

```
#Primitive type
x = 5
y = x
y = 10
print(x) # 5 (x is unchanged)
#Reference type
list1 = [1,2,3]
list2 = list1
list2.append(4)
print(list1) # [1,2,3,4] (list1 changed because list2 references the same object)
```

07*Garbage Collection Meaning:

Garbage Collection: automatic memory management in Python

Objects that no longer have references are automatically deleted by Python

Similar to Java's garbage collector

Example:

```
class Person:
    def __init__(self,name):
        self.name = name
p = Person("Ali")
p = None # The Person object has no references -> eligible for garbage collection
```

08*Using Classes from Python Library:

Python has many build-in classes , similar to Java libraries.

The datetime Class:

```
from datetime import datetime
now = datetime.now()
print(now)
```

The random Class:

```
import random
print(random.randint(1,10)) #Random number between 1 and 10
```

09*Accessing Static vs Non-Static Attributes and Constants:

Non-static attribute : belongs to an object , accessed via object (object.attr)

Static (class) attribute: shared by all objects of the class , accessed via class name (Class.attr)

Example:

```
class Circle :
    pi = 3.14 #static / class attribute
    def __init__(self , radius) :
        self.radius = radius #Non-static / instance attribute
c1 = Circle(5)
c2 = Circle(7)
print(Circle.pi) #3.14 (static)
print(c1.radius) #5
print(c2.radius) #7
```

Output:

```
3.14
5
7
```

10*Array of Objects:

Array of objects : a collection (list) that stores multiple objects

General Formula in Python : objects_list = [Object1 , Object2 , Object3]

Example:

```
class Circle :
    pi = 3.14
    def __init__(self,radius) :
        self.radius = radius
    def area(self):
        return Circle.pi*self.radius**2
#Array (list) of Circle objects:
circles = [Circle(3) , Circle(5) , Circle(7)]
#Accessing each object
for c in circles:
    print("Radius:" , c.radius , "Area" , c.area())
```

Output:

```
Radius: 3 Area: 28.26
Radius: 5 Area: 78.5
Radius: 7 Area: 153.86
```

11*Visibility Modifiers:

Python does not have strict visibility modifiers , but uses conversions:

Public : normal attribute : radius

Private : prefix with `_` or `__` (e.g. , `_pi` or `__pi`)

Example:

```
class Circle:
    def __init__(self , radius):
        self.radius = radius #public
        self.__pi = 3.14 # private
c = Circle(5)
print(c.radius) #5
# print(c.__pi) #Error: private attribute
```

12*Data Field Encapsulation:

Why should data fields be private?

Protects data from accidental modification.

Allows the class to control access and validation.

Improves security and maintainability.

Need for Get and Set Methods

13*Role of Getter and Setter Methods:

Example:

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius #private
    # Getter
    def get_radius(self):
        return self.__radius
    # Setter
    def set_radius(self,r):
        if r>0:
            self.__radius = r
        else:
            print("Radius must be positive!")
c = Circle(5)
print(c.get_radius()) #5
set_radius(10)
print(c.get_radius()) #10
c.set_radius(-3) #Radius must be positive!
```

14*Methods to Output the Data Fields:

Print() : prints values

toString() : returns a string representation of the object

```
class Circle:
    def __init__(self,radius):
        self.__radius = radius
    def __str__(self) # toString() equivalent
        return f"Circle with radius: {self.__radius}"
c= Circle(7)
print(c)
```

15*Passing Objects to Methods (Pass by Reference):

In Python , objects are passed by reference

Modifying an object inside a method changes the original object

```
class Circle :
    def __init__(self , radius):
        self.radius = radius
    def double_radius(c):
        c.radius *=2
c1 = Circle(5)
double_radius(c1)
print(c1.radius)
```

Output:

10

16*The self Reference:

In Python , the keyword self works like this in other languages
It refers to the current object _ the one that's calling the method or constructor

Example using the Circle class:

```
class Circle :
    def __init__(self , radius):
        self.radius = radius # "self" refers to the current object
    def area(self):
        return 3.14*self.radius*self.radius
c = circle(5)
print(c.area()) # Output: 78.5
```

17*What if we do NOT use self when we should:

If you forget to use self , Python will think you are referring to a local variable inside a method , not the instance variable . This can cause errors or wrong results

Example:

```
class Circle:
    def __init__(self , radius) :
        radius = radius # This does nothing
    def show(self):
        print(radius) #Error: radius not defined
```

Correct Version:

```
class Circle:
    def __init__(self , radius):
        self.radius = radius
    def show(self):
        print(self.radius)
c = circle(5)
c.show() # Output: 5
```

18*Object Composition:

Object Composition means that one class contains an object of another class as a field (variable). It helps build complex objects using simpler ones - like a Person having an Address

Example:

```
class Address:
    def __init__(self , city , street):
        self.city = city
        self.street = street
class Person:
    def __init__(self , name , address):
        self.name = name
        self.address = address # object of Address inside Person
    def show_info(self):
        print("Name: " , self.name)
        print("City: " , self.address.city)
        print("Street: " , self.address.street)
addr = Address("Beirut" , "Main Street 12")
P = person("Jawad" , addr)
p.show_info()
```

Chapter 06: Strings

01*Definition of a String:

A string is a sequence of characters.

Strings are immutable in Python (cannot be changed after creating).

Example:

```
s = "Hello , Python!"  
print(s)
```

Output:

```
Hello , Python !
```

02*Constructing a String:

In Python , strings can be created using: string literals or joining a list of characters

A) Using a string literal

```
s1 = "Welcome"  
print(s1)
```

B) Using the str() constructor

```
s2 = str("Welcome")  
print(s2)
```

C) Using a list (array) of characters

```
chars = [ 'W' , 'e' , 'l' , 'c' , 'o' , 'm' , 'e' ]  
s3 = "".join(chars)  
print(s3)
```

Output:

```
Welcome  
Welcome  
Welcome
```

03*Internal String(String Pool Concept):

Strings created with literals may point to the same memory (interning).

Using str() or dynamic creation may create a different object

```
s1 = "Welcome"  
s2 = str("Welcome")  
print(s1 == s2) # True (values are equal)  
print(s1 is s2) # True or False (depends on interning)
```

04*String Comparison:

A) Equals(s1) -> value comparison

```
s1 = "Hello"  
s2 = "hello"  
print(s1 == s2) #False
```

B) EqualsIgnoreCase(s1) -> ignore case

```
print(s1.lower() == s2.lower()) #True
```

C) compareTo(s1) -> lexicographic comparison

```
s1 = "abc"
s2 = "abd"
print((s1 > s2) - (s1 < s2)) # -1 (abc < abd)
```

D) compareToIgnoreCase(s1):

```
s1 = "abc"
s2 = "ABC"
print((s1.lower() > s2.lower()) - (s1.lower() < s2.lower())) # 0 (equal ignoring case)
```

E) regionMatches(start, substring)

```
s1 = "HelloWorld"
print(s1[0:5] == "Hello") # True (match first 5 chars)
```

F) startsWith(prefix):

```
s = "HelloWorld"
print(s.startswith("Hello")) # True
```

G) endsWith(suffix):

```
print(s.endswith("World")) # True
```

Additional Python String Comparison Methods:

in -> check substring existence

```
print("World" in s) # True
```

not in -> checks substring absence

```
print("Python" not in s) # True
```

str.find(substring) -> index of first occurrence or -1

```
print(s.find("World")) # 5
```

str.count(substring) -> number of occurrences

```
print(s.count("l")) # 3
```

05*String Length, Characters, and Combining Strings:

A string in Python is a sequence of characters

To know its length, we use the len() function

To access a specific character, we use square brackets with an index

To combine two strings, we use the + operator.

Example:

```
s = "Welcome"
#Find the number of characters
print(len(s)) # Output : 7
#Access a character by its positive (index starts from 0)
print(s[0]) # Output: W
#Combine two strings
s2 = "Home"
print(s+s2) #Output: WelcomeHome
```

06*Obtaining Substrings:

A substring means a smaller part of the main string

07*Converting , Replacing , and Splitting Strings:

lower() : Converts all characters in a string to lowercase

```
s = "HELLO"  
print(s.lower()) # Output : hello
```

upper() : Converts all characters to uppercase

```
print(s.upper()) # Output : HELLO
```

strip() : Removes all spaces from the beginning and end of the string

```
s = " hi there "  
print(s.strip()) # Output : hi there
```

replace() : Replaces all occurrences of a word or character with another

```
s = "hi hi"  
print(s.replace("hi", "yo")) # Output : yo hi
```

split() : Divides a string into smaller parts (a list) based on a separator (called a delimiter)

If no separator is given, it splits by spaces

```
s = "apple banana grape"  
print(s.split()) # Output: ['apple', 'banana', 'grape']
```

You can also use a specific separator:

```
s = "apple,banana,grape"  
print(s.split(",")) # Output : ['apple', 'banana', 'grape']
```

08*Splitting a String:

Splitting helps you break a sentence into separate words or values.

It's very useful when working with data from files or user input

Example:

```
text = "Python is fun"  
words = text.split()  
print(words) # Output : ['Python', 'is', 'fun']
```

You can then access each word separately:

```
print(words[0]) # Output : Python
```

09*The "." Delimiter:

If you want to split by a dot ".", it's treated like a normal character (not special)

But in regular expressions, it has a special meaning - it means "any character".

To avoid confusion, you can simply do this:

```
s = "www.google.com"  
parts = s.split(".")  
print(parts) # Output : ['www', 'google', 'com']
```

10*Finding a Character or Substring:

To search inside a string , Python provides methods like find() and rfind()

find() : Returns the index of the first appearance of a character or word.If not found , it returns -1

```
s = "Welcome"
print(s.find("e")) # Output : 1
print(s.find("z")) # Output : -1
```

You can also tell it where to start searching:

```
print(s.find("e" , 3)) # Output : 6 (search starts from index 3)
```

rfind() : Searches from the right side (end of the String)

```
s = "Welcome"
print(s.rfind("e")) # Output : 6
```

11*Converting Strings to Numbers:

To convert a numeric string to an integer or a float , we use:

int() : Converts a string to an integer

```
s = "123"
num = float(s)
print(num) # Output: 3.14
```

float() : Converts a string to a floating-point number (decimal)

```
s = "3.14"
num = float(s)
print(num) # Output : 3.14
```

If even one character is different, it returns False.

```
s3 = "hello"
print(s1 == s3) # Output: False
```

Case-Insensitive Comparison:

To compare strings without caring about uppercase/lowercase, convert both to the same case using .lower() or .upper() before comparing.

```
s1 = "Hello"
s2 = "hello"
print(s1.lower() == s2.lower()) # Output: True
```

Relational Comparison (>, <, >=, <=):

These compare strings alphabetically (lexicographically) based on the Unicode value of each character.

```
print("apple" < "banana") # Output: True
print("dog" > "cat") # Output: True
```

Uppercase letters come before lowercase in Unicode, so:

```
print("Apple" < "banana") # Output: True
```

startswith(prefix): Checks if the string begins with a certain word or letter.

```
s = "Hello World"
print(s.startswith("He")) # Output: True
print(s.startswith("Wo")) # Output: False
```

endswith(suffix): Checks if the string ends with a certain word or letter.

```
s = "Hello World"
print(s.endswith("World")) # Output: True
print(s.endswith("ld!")) # Output: False
```

find(substring): Finds the position (index) of the first appearance of a substring. If not found, it returns -1.

```
s = "Python is powerful"
print(s.find("is")) # Output: 7
print(s.find("z")) # Output: -1
```

rfind(substring): Same as find(), but searches from the right side of the string.

```
s = "banana"
print(s.rfind("a")) # Output: 5
```

count(substring): Counts how many times a word or character appears in the string.

```
s = "banana"
print(s.count("a")) # Output: 3
```

in / not in: Checks if a substring exists inside another string.

```
s = "Welcome to Python"
print("Python" in s) # Output: True
print("Java" not in s) # Output: True
```

compareTo-like: (using comparison operators)

Python doesn't have a direct compareTo() method like some languages, but you can simulate it using:

```
s1 = "apple"
s2 = "banana"
if s1 == s2:
    print(0)
elif s1 < s2:
    print(-1)
else:
    print(1)
# Output: -1 (because "apple" comes before "banana")
```

Extra method:

regionMatches (like substring matching):

Python can compare parts of strings using slicing:

```
s1 = "Welcome"
s2 = "HelloWelcome"
# Compare region of s1 and s2
print(s1 == s2[5:12]) # Output: True
```

Extra Helpful String Comparison Tools:

isalpha(): Checks if all characters are letters.

```
print("abc".isalpha()) # Output: True
```

isdigit() : Checks if all characters are digits.

```
print("1234".isdigit()) # Output: True
```

isalnum() : Checks if all are letters or digits.

```
print("abc123".isalnum()) # Output: True
```

islower() / isupper() : Checks if all letters are lowercase or uppercase.

```
print("hello".islower()) # Output: True
```

```
print("HELLO".isupper()) # Output: True
```

Chapter 07 : Inheritance and polymorphism

01*Inheritance Definition:

Inheritance allows one class to get (inherit) the properties and methods of another class. It helps in code reusability and logical organization.

In Python, inheritance is created by placing the parent class in parentheses:

```
class Parent:
    pass
class Child(Parent):
    pass
```

The Child class automatically gets all methods and variables of the Parent.

02*Superclasses and Subclasses:

The superclass (or base class) is the general one.

The subclass (or derived class) is more specific and inherits from the superclass.

Example:

```
class Animal: # Superclass
    def eat(self):
        print("Animal eats")
class Dog(Animal): # Subclass
    def bark(self):
        print("Dog barks")
d = Dog()
d.eat() # Output: Animal eats
d.bark() # Output: Dog barks
```

03*Superclass Constructor Is Always Invoked:

When a subclass is created, Python automatically calls the superclass's constructor first.

If you use super(), you control when it's called.

If not, Python may call a default one.

04*Constructor Chaining:

Constructor chaining means that constructors of all parent classes are executed automatically from top to bottom of the inheritance chain.

Example:

```
class A:
    def __init__(self):
        print("A's constructor")
class B(A):
    def __init__(self):
        super().__init__()
        print("B's constructor")
class C(B):
    def __init__(self):
        super().__init__()
        print("C's constructor")
obj = C()
```

Output:

```
A's constructor
B's constructor
C's constructor
```

05*Example: Superclass Without No-Arg Constructor

If a superclass needs arguments, you must call it with `super()` correctly.

```
class A:
    def __init__(self, x):
        self.x = x
class B(A):
    def __init__(self):
        # super().__init__() # will cause an error (missing argument)
        super().__init__(10) # correct
        print("B created")
b = B()
```

06*Calling Superclass Methods:

You can call a method from the superclass using `super().method_name()`.

```
class A:
    def show(self):
        print("A show")
class B(A):
    def show(self):
        super().show() # calls A's method
        print("B show")
b = B()
b.show()
```

Output :

```
A show
B show
```

07*Rewriting the Circle Class Using super():

```
class GeometricObject:
    def __init__(self, color):
        self.color = color
class Circle(GeometricObject):
    def __init__(self, color, radius):
        super().__init__(color)
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius
c = Circle("Red", 5)
print(c.color) # Output: Red
print(c.area()) # Output: 78.5
```

08*Overriding Methods:

Overriding means redefining a method in the subclass that already exist in the superclass.

```
class Animal:
    def sound(self):
        print("Some sound")
class Dog(Animal):
    def sound(self): # overriding
        print("Bark")
d = Dog()
d.sound() # Output: Bark
```

09*Overloading Methods:

Python doesn't support method overloading directly (same name, different parameters). But you can simulate it using default arguments.

```
class Math:
    def add(self, a, b=0, c=0):
        return a + b + c
m = Math()
print(m.add(5)) # Output: 5
print(m.add(5, 3)) # Output: 8
print(m.add(5, 3, 2)) # Output: 10
```

10*Overloading vs Overriding:

Overloading:

Same method name, different parameters (in same class).

Overriding:

Same method name, same parameters (in subclass).

11*The Object Class and Its str() Method:

In Python, every class automatically inherits from the object class. The str() method defines how the object is shown when printed.

```
class Person:
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return f"Person name: {self.name}"
p = Person("Jawad")
print(p) # Output: Person name: Jawad
```

12*Circle Class (with and without str()):

Without str():

```
class Circle:
    def __init__(self, r):
        self.r = r
c = Circle(5)
print(c)
# Output: <__main__.Circle object at 0x000...> (not readable)
```

With str():

```
class Circle:
    def __init__(self, r):
        self.r = r
    def __str__(self):
        return f"Circle radius: {self.r}"
c = Circle(5)
print(c)
# Output: Circle radius: 5
```

13*Polymorphism Definition:

Polymorphism means "many forms."

It allows you to use one function or method name that behaves differently depending on the object calling it.

14*Polymorphism Application:

```
class Animal:
    def sound(self):
        print("Some sound")
class Dog(Animal):
    def sound(self):
        print("Bark")
class Cat(Animal):
    def sound(self):
        print("Meow")
animals = [Dog(), Cat(), Animal()]
for a in animals:
    a.sound()
```

Output :

```
Bark
Meow
Some sound
```

15*Dynamic Binding:

Dynamic binding means that the method that gets called is decided at runtime, not during compilation. In the example above, each a.sound() call is chosen dynamically based on the object type.

16*Casting Objects:

Python is dynamically typed, so you don't usually need casting.

But you can check types or manually treat an object as a certain type when needed.

17*Why Casting Is Necessary:

Sometimes you may receive an object as a general type, but you know it's actually a subclass. Casting helps you access subclass-specific methods safely.

18*Casting from Superclass to Subclass:

You can use `isinstance()` to verify before accessing subclass features.

```
class Animal:
    pass
class Dog(Animal):
    def bark(self):
        print("Woof!")
a = Dog() # superclass reference
if isinstance(a, Dog):
    a.bark() # Output: Woof!
```

19*The isinstance Operator:

`isinstance(obj, ClassName)` checks if an object is of a given class (or subclass).

```
class A:
    pass
class B(A):
    pass
b = B()
print(isinstance(b, B)) # Output: True
print(isinstance(b, A)) # Output: True (B inherits from A)
```

Chapter 08 : ArrayList

01*The ArrayList Class:

In Python, we can store multiple values inside one variable using a list.

A list can grow or shrink while the program is running — we can add, remove, or change elements easily.

Syntax:

```
my_list = [] # empty list
my_list = [1, 2, 3] # list with elements
```

Example :

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

Output :

```
['apple', 'banana', 'cherry']
```

02*The ArrayList Methods:

`append()` : Adds an element to the end of the list.

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)
```

Output :

```
['apple', 'banana', 'cherry']
```

insert() : Inserts an element at a specific index.

```
fruits = ["apple", "cherry"]  
fruits.insert(1, "banana")  
print(fruits)
```

Output :

```
['apple', 'banana', 'cherry']
```

remove() : Removes the first element that matches the given value.

```
fruits = ["apple", "banana", "cherry"]  
fruits.remove("banana")  
print(fruits)
```

Output :

```
['apple', 'cherry']
```

pop() : Removes and returns an element at a specific index.

If you don't specify an index, it removes the last element.

```
numbers = [10, 20, 30]  
x = numbers.pop(1)  
print(numbers)  
print("Removed:", x)
```

Output:

```
[10, 30]  
Removed: 20
```

clear() : Removes all elements from the list.

```
fruits = ["apple", "banana"]  
fruits.clear()  
print(fruits)
```

Output :

```
[]
```

index() : Finds the position of the first matching element

```
numbers = [5, 10, 15, 10]  
print(numbers.index(10))
```

Output :

```
1
```

count() : Counts how many times a specific element appears.

```
numbers = [1, 2, 2, 3, 2]  
print(numbers.count(2))
```

Output :

```
3
```

sort() : Sorts the list in ascending order.

```
numbers = [4, 1, 3, 2]
numbers.sort()
print(numbers)
```

Output :

```
[1, 2, 3, 4]
```

reverse() : Reverses the order of the list.

```
numbers = [1, 2, 3, 4]
numbers.reverse()
print(numbers)
```

Output :

```
[4, 3, 2, 1]
```

copy() : Creates a duplicate of the list.

```
a = [1, 2, 3]
b = a.copy()
b.append(4)
print("a:", a)
print("b:", b)
```

Output :

```
a: [1, 2, 3]
b: [1, 2, 3, 4]
```

03*The ArrayList as a Generic Class:

In Python, a list can hold values of any type — numbers, strings, or even other lists. This makes it very flexible.

```
mixed = [1, "apple", 3.5, True]
print(mixed)
```

Output

```
[1, 'apple', 3.5, True]
```

If you want, you can write a "type hint" just to make it clearer:

```
numbers: list[int] = [1, 2, 3]
```

Non-Generic vs Generic:

Python doesn't force you to use one data type in a list.

But sometimes, for clarity, you can write code that expects only numbers or only strings.

Example (mixed list):

```
data = [1, "hello", 3.5]
print(data)
```

Example (list of integers):

```
def print_double(values: list[int]):
    for v in values:
        print(v * 2)
print_double([1, 2, 3])
```

Output:

```
2
4
6
```

05*Simplified way to create and use lists:

You don't need any special class — just create a list and start using it.

```
students = []
students.append("Ali")
students.append("Sara")
print(students)
```

Output:

```
['Ali', 'Sara']
```

06*Difference between an Array and a List:

An array can only store elements of one specific type and has a fixed size.

A list can store elements of any type and can grow or shrink freely.

Example using an array:

```
import array
nums = array.array('i', [1, 2, 3])
print(nums)
```

Output:

```
array('i', (1, 2, 3))
```

Example using a list:

```
nums = [1, 2, 3]
nums.append(4)
print(nums)
```

Output:

```
[1, 2, 3, 4]
```

Chapter 09: Exception Handling and Text I/O

01*Introduction:

When a program runs, sometimes unexpected problems happen — like dividing by zero, opening a missing file, or giving invalid input.

These problems are called exceptions.

In Python, exceptions are managed using try, except, and sometimes finally blocks.

This helps the program handle errors gracefully instead of crashing.

02*Exception Handling Overview:

The idea of exception handling is simple:

You try to execute a piece of code that might cause an error.

If an error happens, it is caught in an except block.

You can also use finally to execute code that runs no matter what (like closing a file).

General formula:

```
try:
    # risky code
except SomeErrorType:
    #what to do if error happens
finally:
    #code that always run
```

03*Throwing and Catching an Exception:

try-except Example:

```
try:
    x = int(input("Enter a number:"))
    print("Result:",10/x)
except ZeroDivisionError:
    print("Error:Cannot divide by zero!")
except ValueError:
    print("Error: Invalid input! Please enter a number.")
```

Output (if user enters 0):

Error: Cannot divide by zero !

04*Exception Types:

Each error type in Python is represented by a class

Some common ones include:

ZeroDivisionError

ValueError

IndexError

TypeError

FileNotFoundError

KeyError

You can catch one or multiple exceptions as needed

05*Some Important Exceptions:

Let's look at a few that appear often in real program

ValueError:

Occurs when a value is the wrong type

```
try:
    x = int("abc")
except ValueError:
    print("Cannot convert text to number!")
```

Output:

Cannot convert text to number!

IndexError:

Happens when you try to access an element outside the list range

```
numbers = [10,20,30]
try:
    print(numbers[5])
except IndexError:
    print("Index out of range!")
```

Output:

```
Index out of range!
```

ZeroDivisionError:

```
try:
    print(10/0)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output:

```
Cannot divide by zero!
```

06*Handling None and Missing Data:

None represents the absence of a value (similar to "null" in other languages)
If you try to access a method or property of None , you'll get a TypeError.

Example:

```
name = None
try:
    print(name.upper())
except AttributeError:
    print("Cannot use a method on None!")
```

Output:

```
Cannot use a method on None!
```

07*Illegal Argument Example:

We can manually raise an error if an invalid argument is passed to a function.

```
def set_age(age):
    if age<0:
        raise ValueError("Age cannot be negative!")
    print("Age set to:",age)
try:
    set_age(-5)
except ValueError as e:
    print("Error:",e)
```

Output:

```
Error : Age cannot be negative!
```

08*Checked vs Unchecked Exceptions:

All exceptions are unchecked. This means you don't need to declare them before writing your function , but you should handle them to keep your program safe

09*The Exception Class:

All errors in Python are subclasses of the built-in Exception class

You can even create your own exception by making a class that inherits from it

Example:

```
class MyError(Exception):
    pass
try:
    raise MyError("Something went wrong!")
except MyError as e:
    print(e)
```

Output:

```
Something went wrong!
```

10*Declaring and Throwing Exceptions:

We use the raise keyword to throw (raise) an exception manually

Example:

```
x = -1
if x < 0:
    raise Exception("x cannot be negative")
```

Output:

```
Exception: x cannot be negative
```

11*Using finally Block:

The finally block always executes - whether an error happens or not

It's often used to close files or release resources

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found!")
finally:
    print("Closing file...")
```

Output (if file missing) :

```
File not found!
Closing file
```

12*Working with Files (Text I/O):

Opening and Reading a File

```
try:
    file = open("output.txt", "w")
    file.write("Hello ,Jawad!\nWelcome to Python I/O.")
    print("File written successfully!")
finally:
    file.close()
```

Output (file content in output.txt)

```
Hello ,Jawad!
Welcome to Python I/O
```

Appending to a File:

With open("output.txt", "a") as file:

```
File.write("\nThis line was added later.")
```

Output (file content):

```
Hello, Jawad!  
Welcome to Python I/O  
This line was added later.
```

Using 'with' Keyword (Recommended Way)

Using with automatically closes the file even if an error happens

```
try:  
    with open("data.txt", "r") as file:  
        for line in file:  
            print(line.strip())  
except FileNotFoundError:  
    print("Error: File not found.")
```

Chapter 10 : Abstract Classes and Interfaces

01*Abstract Classes:

Use abc.ABC + @abstractmethod to define a class that cannot be instantiated and that forces subclasses to implement certain methods

An abstract class declares one or more methods without implementation (abstract methods) . It can also contain concrete(implemented) methods and fields. You cannot create an object of an abstract class; you must subclass it and implement the abstract methods

Python pattern:

```
from abc import ABC, abstractmethod  
class MyBase(ABC):  
    @abstractmethod  
    def my_method(self):  
        pass
```

Example:

```
from abc import ABC, abstractmethod  
class Animal(ABC):  
    @abstractmethod  
    def sound(self):  
        pass  
#Trying to instantiate fails:  
# a = Animal() #Type Error: Can't instantiate abstract class Animal with abstract methods sound  
class Dog(Animal):  
    def sound(self):  
        return "Bark"  
d = Dog()  
print(d.sound())
```

Output:

```
Bark
```

02*Abstract class GeometricObject:

Define common attributes and the abstract API (e.g. area() , perimeter()) in the abstract class;
concrete shapes implement them

Example:

```
from abc import ABC, abstractmethod
import math

class GeometricObject(ABC):
    def __init__(self,color="white"):
        self.color = color #concrete field

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def info(self): # concrete helper method
        return f"Color:{self.color}"

class Circle(GeometricObject):
    def __init__(self , radius , color="white"):
        super().__init__(color)
        self.radius = radius

    def area(self):
        return math.pi*self.radius**2

    def perimeter(self):
        return 2 * math.pi * self.radius

class Rectangle(GeometricObject):
    def __init__(self , width , height , color = "white"):
        super().__init__(color)
        self.width = width
        self.height = height

    def area(self):
        return self.width*self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

c = Circle(5 , color = "red")
r = Rectangle(3 , 4 , color = "blue")
print(c.info() , "Area: " , round(c.area() , 2) , "Perimeter:" , round(c.perimeter() , 2))
print(r.info() , "Area: " , r.area() , "Perimeter: " , r.perimeter())
```

Output:

```
Color: red Area : 78.54 Perimeter: 31.42
Color: blue Area : 12 Perimeter: 14
```

03*Why Abstract Methods?

Use abstract methods to enforce a contract: every subclass must implement these behaviors
Abstract methods ensure that all subclasses share the same interface . They help design correct polymorphic code (you can treat different subclasses uniformly)

Mini example : treat different shapes uniformly:

```
shapes = [Circle(2) , Rectangle(2,5)]
for s in shapes:
    print(type(s).__name__, "area=",round(s.area(),2))
```

Output:

```
Circle area = 12.57
Rectangle area = 10
```

04*Interesting points about Abstract Classes:

An abstract class can have concrete methods and fields.

You cannot instantiate it.

You can have abstract @property , @classmethod , @staticmethod with @abstractmethod.

A subclass must implement all abstract methods(unless it is abstract too)

Multiple inheritance with ABCs is allowed . Example (abstract property & classmethod):

```
from abc import ABC , abstractmethod
class Base(ABC):
    @property
    @abstractmethod
    def name(self):
        pass
    @classmethod
    @abstractmethod
    def create_default(cls):
        pass
class Impl(Base):
    def __init__(self , name):
        self._name = name
    @property
    def name(self):
        return self._name
    @classmethod
    def create_default(cls):
        return cls("default")
i = Impl.create_default()
print(i.name)
```

Output:

```
Default
```

05*Interfaces (Python style):

Use typing Protocol for structural (duck-typed) interfaces or abc.ABC for nominal interfaces. Protocols specify method/attribute signatures without implementation

Python has no separate interface keyword . You can declare an interface by creating an abstract base class (ABC) with only abstract methods , or use Protocol (PEP 544) to express that any object matching the shape is acceptable.

Example:

```
from typing import Protocol
class Comparable(Protocol):
    def __lt__(self, other) -> bool: ...
    def __eq__(self, other) -> bool: ...
class Person:
    def __init__(self, age):
        self.age = age
    def __lt__(self, other):
        return self.age < other.age
people = [Person(30), Person(20), Person(25)]
people.sort() # works because Person implements __lt__ and __eq__
print([p.age for p in people])
```

Output:

```
[20,25,30]
```

06*Omitting Modifiers in Interfaces:

There are no access modifiers (public/private) in interface definitions - all methods are meant to be implemented and are effectively public

In Python, methods declared in ABCs or Protocols are publicly visible. Privacy is by convention (`_name` or `__name`), but interface methods are not private

Example:

Define class `MyInterface(Protocol)`: `def do-it(self): ...` - implementers provide `do_it` publicly.

07*The Comparable idea(what Comparable provides):

Implement rich comparison dunder methods (`__lt__`, `__le__`, `__eq__`, etc.) or use `functools.total_ordering` to get full ordering from minimal definitions

Example with `total_ordering`:

```
from functools import total_ordering
@total_ordering
class Box:
    def __init__(self, volume):
        self.volume = volume
    def __eq__(self, other):
        return self.volume == other.volume
    def __lt__(self, other):
        return self.volume < other.volume
boxes = [Box(30), Box(10), Box(20)]
boxes.sort()
print([b.volume for b in boxes])
```

Output:

```
[10,20,30]
```

08*Interfaces vs Abstract Classes - which to use ?

Use an abstract class (ABC) when you want to supply some shared implementation or state(fields)
Use a Protocol (structural interface) when you only need to specify a method/attribute shape and allow any compatible object to be used (duck typing)

Differences:

ABC : nominal , can hold fields and concrete code , enforces subclassing

Protocol : structural , checks "does this object have the methods?" (no need to inherit)

Small illustrative example:

If you want to share a helper method describe() across shapes use ABC with concrete describe . If you only require objects to be "sortable" accept any object with __lt__ (Protocol or duck typing).

Notes:

Abstract class : class X(ABC): @abstractmethod def m(self): ... -> cannot instantiate

Concrete subclass : implement all abstract methods

Protocol (interface) : specify method signatures ; any object with those methods qualifies.

Comparable: implement __lt__ and __eq__ (or use total_ordering).

When to use : ABC when sharing code/state; Protocol when only the API shape matters

Chapter 11: Inner and Nested Classes

01*Definition:

A nested class is a class defined inside another class.

It helps group classes that are logically related together and limit their visibility to the outer class.

When a class is defined inside another, it's called a nested class.

The outer class acts as a container, and the inner class is accessible through it.

This is useful when the inner class logically belongs only to the outer class, and you don't need it to be used elsewhere.

Example:

```
class Outer:
    def __init__(self):
        self.name = "Outer class"
    class Inner:
        def message(self):
            print("This is an Inner class inside Outer")
# Creating objects
outer = Outer()
inner = outer.Inner()
inner.message()
```

Output:

```
This is an Inner class inside Outer
```

02*Types of Nested Classes:

Python doesn't officially divide nested classes into categories like some other languages, but conceptually, we can think of four main types depending on where and how the nested class is defined or used.

(a) Member Inner Class

A member inner class is a class defined directly inside another class and can access the outer class's attributes.

This is the most common form — a class that lives inside another class and works closely with its data.

Example:

```
class Person:
    def __init__(self, name):
        self.name = name
    class Address:
        def __init__(self, city):
            self.city = city
# Accessing nested class
p = Person("Jawad")
a = p.Address("Beirut")
print(p.name, "lives in", a.city)
```

Output :

```
Jawad lives in Beirut
```

(b) Static Nested Class (Simulated)

A static nested class doesn't depend on an instance of the outer class.

Python doesn't have the keyword "static" for nested classes, but you can simulate the same behavior by defining the inner class normally inside the outer class and accessing it through the class name rather than an instance.

Example:

```
class MathTools:
    class Calculator:
        def add(x, y):
            return x + y
# Access directly through the outer class name (no instance)
print(MathTools.Calculator.add(3, 7))
```

Output:

```
10
```

(c) Local Inner Class

A local inner class is a class defined inside a method (function) of another class.

You can define a class within a method — it's only visible and usable inside that method.

Useful when you need a helper class for a specific task.

Example:

```
class Outer:
    def show(self):
        class LocalInner:
            def message(self):
                print("This is a local inner class inside a method")
        obj = LocalInner()
        obj.message()
outer = Outer()
outer.show()
```


Output :

This is a local inner class inside a method

(d) Anonymous Inner Class (Simulated)

An anonymous inner class is created without a name, usually used once for a specific purpose. Python doesn't support anonymous classes directly, but you can simulate them using lambda functions or `type()` to create a class dynamically on the fly.

Example using `type()`:

```
# Creating a temporary inner class dynamically
TempClass = type(
    "Temp",
    (),
    {
        "message": lambda self: print(
            "Hello from anonymous inner class!"
        )
    }
)
obj = TempClass()
obj.message()
```

Output :

Hello from anonymous inner class!

Example using lambda (for single-use behavior):

```
def outer():
    # Anonymous function used as a single-use inner behavior
    display = lambda: print("This acts like an anonymous inner class")
    display()
outer()
```

Output:

This acts like an anonymous inner class

Chapter 12 : Enum

01*Definition:

Enum (short for Enumeration) is a special data type used to define a set of named constant values. It's useful when you have a fixed group of related constants (like days of the week, directions, months, etc.). In Python, Enum is provided by the enum module.

Example:

```
from enum import Enum
class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3
print(Color.RED)
print(Color.RED.value)
```

Output:

```
Color.RED
1
```

02*Features of Enum:

Enum is used to create named constants.

Enum values are unique.

Each constant is an object of the Enum class.

Enums improve code readability.

You can compare Enum members.

You can loop through Enum values.

```
from enum import Enum
class Day(Enum):
    MONDAY = 1
    TUESDAY = 2
    WEDNESDAY = 3
for d in Day:
    print(d)
```

Output:

```
Day.MONDAY
Day.TUESDAY
Day.WEDNESDAY
```

Accessing Enum Values:

```
print(Day.MONDAY.name)
print(Day.MONDAY.value)
```

Output:

```
MONDAY
1
```

03*Example: (Python)

```
from enum import Enum
class Direction(Enum):
    NORTH = 1
    SOUTH = 2
    EAST = 3
    WEST = 4
dir = Direction.EAST
print("Direction:", dir.name)
if dir == Direction.NORTH:
    print("You are going up!")
elif dir == Direction.EAST:
    print("You are going right!")
```

Output:

```
Direction: EAST
You are going right!
```

04*Enum with Fields and Methods (Python)

An Enum can have values and methods.

```
from enum import Enum
class Day(Enum):
    MONDAY = "Start of week"
    FRIDAY = "End of work week"
    SUNDAY = "Rest day"
for d in Day:
    print(d.name, ":", d.value)
```

Output:

```
MONDAY : Start of week
FRIDAY : End of work week
SUNDAY : Rest day
```

05* Python Enum - Key Points:

Python Enum is available in the built-in enum module.

Each Enum constant is an object.

Enum prevents using invalid values.

You can define methods and properties inside an Enum.

You can iterate over all values using a for loop.

Example:

```
from enum import Enum
class Color(Enum):
    RED = 1
    BLUE = 2
    GREEN = 3
for color in Color:
    print(color)
```

Output:

```
Color.RED
Color.BLUE
Color.GREEN
```

Enum is Type-safe:

It means you cannot assign a value that is not part of the Enum.

Example:

```
color = Color.RED  
color = 5 # Error
```

Using Enum with if-elif (Python)

Python does not have switch statements, so we use if-elif.

```
if color == Color.RED:  
    print("Red")  
elif color == Color.BLUE:  
    print("Blue")
```

Access Enum name and value:

```
print(Color.RED.name)  
print(Color.RED.value)
```

Output:

```
RED  
1
```